LEMBAR KERJA PRAKTIKUM (LKP)

Pertemuan 4 — Basis Data, Migrasi, Eloquent & Seeder

Tugas Mandiri: Implementasi Database Sesuai Proyek Kelompok

Pemrograman Web Lanjut · Universitas Hasyim Asy'ari Tebuireng Jombang · Program Studi Teknik Informatika · Semester V

Mata Kuliah	Pemrograman Web Lanjut (3 SKS — Semester V)
Dosen Pengampu	Edwin Hari Agus Prastyo, S.Kom., M.Kom.
Sifat Tugas	Mandiri (Individu) — Berdasarkan Studi Kasus Proyek Kelompok
Teknologi	Laravel 11/12 · PHP 8.2+ · Composer · Laragon (MySQL/MariaDB) · Eloquent ORM
CPMK	CPMK-1, CPMK-2 · **Sub-CPMK:** S2, S4
Prasyarat	Selesai Modul 03 (Arsitektur Laravel, MVC, dan Perancangan Diagram UML)
Luaran Utama	Diagram ERD + File Migrasi + Model Eloquent Berelasi + Factory & Seeder + Data Dummy
Penyerahan Kode	Source code ditulis pada proyek Laravel lalu di-*commit* ke repositori Git — **tidak disalin ke lembar kerja ini**

Copyright © Edwin Hari Agus Prastyo, S.Kom., M.Kom.

I. IDENTITAS MAHASISWA & PROYEK

Setiap mahasiswa wajib mengisi data diri dan identitas proyek kelompok yang telah disepakati:

Kelas / Semester	A / V (Lima)
Nama Lengkap	Hanifatul Khoir, Nurul Lailatus Solekha, Dimas Ferdiansyah
NIM **Nama Kelompok**	2495114001, 2495114003, 2495114036 Kelompok 1
Judul Proyek Web	Aplikasi JAHITYUK!!
Kasus Proyek Kelompok	Kasus JAHITYUK!! — Aplikasi Pemesanan dan Antrean Jasa Jahit
URL Repositori Git	`https://github.com/_____`
Branch Pengerjaan	main

II. CAPAIAN & TUJUAN PRAKTIKUM

Setelah menyelesaikan lembar kerja praktikum mandiri ini, mahasiswa mampu:

1. Mengonfigurasi koneksi basis data **MySQL/MariaDB melalui Laragon** pada berkas `.env`.
2. Merancang **Entity Relationship Diagram (ERD)** yang selaras dengan rancangan *Class Diagram* dari Modul 03.
3. Membuat dan menyusun urutan file **migrasi** tabel beserta konstrain integritas data.
4. Mendefinisikan **Model Eloquent** dan mengimplementasikan relasi antar entitas.
5. Membangun data dummy realistis menggunakan kombinasi **Factory (Faker)** dan **Database Seeder**.
6. Menjalankan perintah migrasi dan memverifikasi konsistensi struktur data pada DBMS.

> **Keterkaitan CPMK:**
> - **CPMK-1 (Sub-CPMK S2):** Merancang model data dan keterhubungan entitas aplikasi web.
> - **CPMK-2 (Sub-CPMK S4):** Mengintegrasikan basis data, relasi skema, migrasi, dan populasi data awal.

III. RINGKASAN TEORI PENDUKUNG

1. Konvensi Standar Laravel Eloquent

- **Nama Tabel:** Bentuk jamak *snake_case* (contoh: `users`, `products`, `medical_records`).
- **Nama Model:** Bentuk tunggal *PascalCase* (contoh: `User`, `Product`, `MedicalRecord`).
- **Foreign Key:** Nama model tunggal + `_id` (contoh: `user_id`, `category_id`, `doctor_id`).
- **Mass Assignment:** Kolom yang dapat diisi wajib didaftarkan pada properti `$fillable`.

2. Jenis Relasi Eloquent

Jenis Relasi	Method Model Induk	Method Model Anak	Penempatan FK
One-to-Many	`hasMany(Child::class)`	`belongsTo(Parent::class)`	FK di tabel anak
One-to-One	`hasOne(Child::class)`	`belongsTo(Parent::class)`	FK di tabel anak
Many-to-Many	`belongsToMany(Other::class)`	`belongsToMany(Other::class)`	Tabel pivot terpisah

3. Urutan Eksekusi Migrasi

Tabel induk (*parent/master*) **wajib dibuat lebih dahulu** sebelum tabel anak (*child/detail*). Jika urutan terbalik, konstrain `foreignId()->constrained()` akan gagal dengan error *foreign key constraint fail*.

IV. ALAT, BAHAN & LINGKUNGAN KERJA

Pastikan seluruh komponen berikut tersedia dan aktif sebelum memulai:

Kebutuhan	Software / Komponen	Status Kesiapan
Web Server & Database	Laragon + MySQL/MariaDB	[ ] Aktif
Runtime & Package Manager	PHP >= 8.2 & Composer 2.x	[ ] Terpasang
IDE / Code Editor	Visual Studio Code / PHPStorm	[ ] Siap
Database Client	phpMyAdmin / HeidiSQL / DBeaver	[ ] Siap
Diagramming Tool	Draw.io / dbdiagram.io / PlantUML	[ ] Siap
Proyek Laravel	Proyek hasil pengerjaan LKP Modul 03	[ ] Siap berjalan

V. LEMBAR KERJA MAHASISWA MANDIRI

Ketentuan Pengerjaan Lembar Kerja:

- Lembar kerja ini **bukan tempat melampirkan source code**. Seluruh kode (migrasi, model, factory, seeder, konfigurasi `.env`) ditulis pada proyek Laravel kelompok Anda dan di-*commit* ke repositori Git.
- Isi lembar kerja sebatas **rancangan, tabel isian, dan refleksi**. Untuk setiap berkas kode cukup dicatat **path/nama berkas** beserta status *commit*-nya.
- Tautan repositori dan branch pengerjaan dicatat pada Bagian I agar dapat diperiksa dosen/asisten pengampu.
- Kolom **Commit** pada tabel isian ditandai `[X]` setelah berkas di-*commit* dan di-*push* ke repositori; biarkan kosong bila belum.

TAHAP 1: PERANCANGAN ENTITAS & RELASI (ERD)

1.1 Identifikasi Entitas / Model Proyek

Tuliskan seluruh entitas yang akan dibangun pada proyek kelompok Anda, sesuai *Class Diagram* dari Modul 03:

No	Nama Entitas (Model)	Nama Tabel (Database)	Fungsi & Peran dalam Sistem
1	`User`	`users`	Autentikasi dan identitas pengguna sistem
2	Pelanggan	pelanggans	Menyimpan data pelanggan yang menggunakan layanan jasa jahit
3	Penjahit/Karyawan	karyawans	Menyimpan data penjahit/karyawan yang mengelola dan memproses pesanan
4	Admin/Pemilik	admins	Mengelola data sistem, pesanan, pembayaran, aktor, dan laporan
5	Antrean	antreans	Mengelola nomor antrean pelanggan dan status antrean
6	Pesanan	pesanans	Menyimpan data pesanan jasa jahit, ukuran, model, bahan, warna, dan status pesanan
7	Pembayaran	pembayarans	Menyimpan data pembayaran dan bukti pembayaran dari pelanggan
8	Pesan	pesans	Menyimpan komunikasi atau pesan antara pengguna terkait pesanan
9	Laporan	laporans	Menyimpan dan menyediakan data laporan pesanan dan pembayaran

1.2 Matriks Pemetaan Relasi Antar Entitas

Petakan hubungan antar entitas beserta kardinalitas dan foreign key yang digunakan:

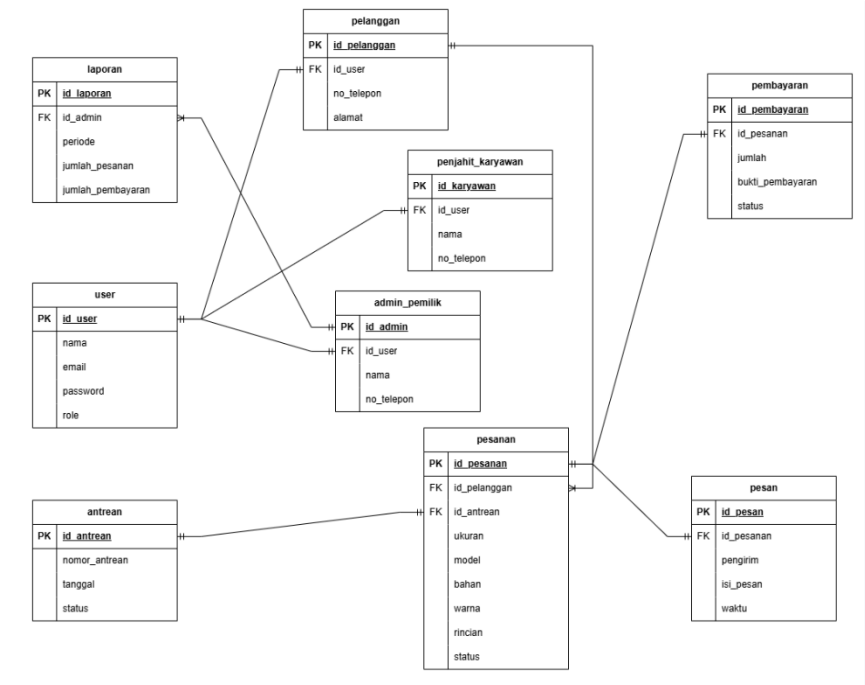
No	Entitas Asal	Bentuk Relasi	Entitas Target	Kolom Kunci Tamu (Foreign Key)
1	User	1 : 1	Pelanggan	pelanggans.id_user
2	User	1 : 1	Karyawan	karyawans.id_user
3	User	1 : 1	Admin	admins.id_user
4	Pelanggan	1 : N	Antrean	antreans.id_pelanggan
5	Pelanggan	1 : N	Pesanan	pesanans.id_pelanggan
6	Karyawan	1 : N	Pesanan	pesanans.id_karyawan
7	Admin	1 : N	Laporan	laporans.id_admin
8	Antrean	1 : 1	Pesanan	pesanans.id_antrean
9	Pesanan	1 : N	Pesan	pesans.id_pesanan
10	Pesanan	1 : 1	Pembayaran	pembayarans.id_pesanan

1.3 Diagram Hubungan Entitas (ERD)

Rancang ERD menggunakan draw.io / dbdiagram.io / PlantUML, lalu lampirkan hasilnya di sini:

STRUKTUR / DIAGRAM — TEMPAT PENEMPELAN GAMBAR






```
+------------------------------------------------------------------------
```

```
Path berkas gambar ERD : screenshots/erd_proyek.png
Tools yang digunakan   : draw.io
```

TAHAP 2: KONFIGURASI DATABASE & ENVIRONMENT

2.1 Pembuatan Database pada DBMS

1. Buka Laragon → **Database** → buka phpMyAdmin
2. Buat basis data baru:

- Nama Database : jahityuk_db
- Collation : `utf8mb4_unicode_ci`

2.2 Konfigurasi Berkas `.env`

Terapkan baris konfigurasi database berikut pada berkas `.env` proyek (berkas `.env` bersifat lokal dan **tidak** di-*commit*; cukup sertakan `.env.example` pada repositori):

KONFIGURASI BERKAS .ENV

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=jahityuk_db
DB_USERNAME=root
DB_PASSWORD=
```

2.3 Uji Koneksi Database

Jalankan perintah berikut di terminal proyek:

PERINTAH TERMINAL (ARTISAN)

```
php artisan migrate:status
```

Tuliskan output terminal atau catatan hasil pengujian:

AREA JAWABAN — SILAKAN TULIS DI SINI

```
Migration name ..................................................... Batch / Status
2026_09_28_135154_create_users_table.....................................[1] Ran
2026_09_28_135355_create_pelanggan_table ................................[1] Ran
2026_09_28_135446_create_karyawan_table .................................[1] Ran
2026_09_28_135512_create_admin_table ....................................[1] Ran
2026_09_28_135538_create_antrian_table ..................................[1] Ran
2026_09_28_135600_create_pesanan_table ..................................[1] Ran
2026_09_28_135623_create_pembayaran_table ...............................[1] Ran
2026_09_28_135717_create_pesan_table ....................................[1] Ran
2026_09_28_135735_create_laporan_table ..................................[1] Ran
```

Status koneksi: [Berhasil] Terhubung Berhasil / [ ]
Terjadi Kendala: ____

TAHAP 3: IMPLEMENTASI SKEMA MIGRASI

3.1 Berkas Migrasi di Repositori

Perintah `make:migration` dijalankan pada terminal proyek, lalu berkas hasilnya di-*commit* ke repositori. Catat berkas migrasi yang Anda hasilkan, terurut dari tabel master ke tabel anak/transaksi:

No	Berkas Migrasi (`database/migrations/`)	Tabel yang Dibuat	Commit
1	2026_09_28_135154 _create_users_table.pl	users	[iya]

2	2026_09_28_135355 _create_pelanggan_ta	pelanggan	[iya]
3	2026_09_28_135446 _create_karyawan_tab	karyawan	[iya]
4	2026_09_28_135512 _create_admin_table.p	admin	[iya]
5	2026_09_28_135538 _create_antrian_table.	antrian	[iya]
6	2026_09_28_135600 _create_pesanan_table	pesanan	[iya]
7	2026_09_28_135623 _create_pembayaran_	pembayaran	[iya]
8	2026_09_28_135717 _create_pesan_table.p	pesan	[iya]
9	2026_09_28_135735 _create_laporan_table	laporan	[iya]

3.2 Skema Tabel Master Inti (Tabel 1)

- **Nama Tabel:** Users
- **Berkas Migrasi:** database/migrations/2026_09_28_135154_create_users_table.php
- **Status Commit:** [ ] Ter-commit

Implementasikan method `up()` pada berkas migrasi tersebut di repositori, lalu jelaskan rancangan kolomnya di sini:

Penjelasan kolom utama tabel ini:

Nama Kolom	Tipe Data	Nullable	Keterangan / Aturan
Id_user	Primary key	tidak	Primary Key (ID unik pengguna)
Nama	Varchar	tidak	Menyimpan teks nama lengkap pengguna
Email	Varchar	tidak	Menyimpan alamat email pengguna (unik)
Password	Varchar	tidak	Menyimpan string kata sandi pengguna
Role	Varchar	tidak	Menentukan hak akses atau pera

3.3 Skema Tabel Transaksi / Detail dengan Foreign Key (Tabel 2)

- **Nama Tabel:** pelanggan
- **Berkas Migrasi:** database/migrations/2026_09_28_135355_create_pelanggan_table.php
- **Merujuk ke Tabel:** _____ dan _____
- **Status Commit:** [ ] Ter-commit

Definisikan foreign key pada berkas migrasi di repositori menggunakan `foreignId()->constrained()` (tambahkan `cascadeOnDelete()` bila data anak harus ikut terhapus). Petakan kolom dan relasinya di sini:

Nama Kolom	Tipe Data	Foreign Key / Constraint	Keterangan
id_pelanggan	primary key	tidak	Primary Key (ID unik pelanggan)
id_user	Foreign key	tidak	Foreign Key yang terhubung ke tabel `users` kolom `id_user` dengan aturan `onDelete('cascade')`
No_telepon	varchar	tidak	Menyimpan nomor telepon atau kontak pelanggan
alamat	text	tidak	Menyimpan informasi teks alamat lengkap pelanggan

3.4 Skema Tabel Tambahan / Pivot (Tabel 3 dan Seterusnya)

(Salin tabel isian ini untuk setiap tabel tambahan sesuai ERD proyek; kodenya tetap berada di repositori)

- **Nama Tabel:** karyawan
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135446_create_karyawan_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
id_karyawan	Primary key	tidak	Primary Key (ID unik karyawan)
id_user	Foreign key	tidak	Foreign Key yang terhubung ke tabel users kolom id_user dengan aturan onDelete('cascade')
nama	Varchar	tidak	Menyimpan teks nama lengkap karyawan
No_telepon	Varchar	tidak	Menyimpan nomor telepon atau kontak karyawan

- **Nama Tabel:** Admin
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135512_create_admin_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
id_admin	Primary key	tidak	Primary Key (ID unik admin)
Id_user	Foreign key	tidak	oreign Key yang terhubung ke tabel `users` kolom `id_user` dengan aturan `onDelete('cascade')`
nama	Varchar	tidak	Menyimpan teks nama lengkap admin
No_telepon	Varchar	tidak	Menyimpan nomor telepon atau kontak admin

- **Nama Tabel:** antrian
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135538_create_antrian_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
Id_antrean	Primary key	tidak	Primary Key (ID unik antrean)
Nomor_antrean	Varchar	tidak	Menyimpan string atau nomor antrean
Tanggal	date	tidak	Menyimpan informasi tanggal antrean
Status	Varchar	tidak	Menyimpan status keterangan antrean

- **Nama Tabel:** pesanan
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135600_create_pesanan_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
Id_pesanan	Primary key	tidak	Primary Key (ID unik pesanan)
Id_pelanggan	Foreign key	tidak	Foreign Key yang terhubung ke tabel `pelanggan` kolom `id_pelanggan` dengan aturan `onDelete('cascade')`
Id_antrean	Foreign key	Ya	Foreign Key yang terhubung ke tabel `antrean` kolom `id_antrean` dengan aturan `onDelete('set null')`
Ukuran	Varchar	tidak	Menyimpan informasi ukuran pesanan
Model	Varchar	tidak	Menyimpan informasi model pesanan
Bahan	Varchar	tidak	Menyimpan informasi bahan yang digunakan
Warna	Varchar	tidak	Menyimpan informasi warna pesanan
Rincian	Text	tidak	Menyimpan rincian atau catatan lengkap pesanan

Status	Varchar	tidak	Menyimpan status terkini dari pesanan

- **Nama Tabel:** pembayaran
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135623_create_pembayaran_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
Id_pembayaran	Primary key	tidak	Primary Key (ID unik pembayaran)
Id_pesanan	Foreign key	tidak	Foreign Key yang terhubung ke tabel `pesanan` kolom `id_pesanan` dengan aturan `onDelete('cascade')`
Jumlah	Decimal	tidak	Menyimpan nominal angka jumlah pembayaran (12 digit, 2 desimal)
Bukti_pembayaran	Varchar	tidak	Menyimpan nama file atau path jalur penyimpanan bukti pembayaran
Status	Varchar	tidak	Menyimpan status terkini dari pembayaran

- **Nama Tabel:** pesan
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135717_create_pesan_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
Id_pesan	Primary key	tidak	Primary Key (ID unik pesan)
Id_pesanan	Foreign key	tidak	Foreign Key yang terhubung ke tabel `pesanan` kolom `id_pesanan` dengan aturan `onDelete('cascade')`
Pengirim	Varchar	tidak	Menyimpan informasi pihak pengirim pesan
Isi_pesan	Text	tidak	Menyimpan teks isi atau konten pesan
Waktu	Timestamp	tidak	Menyimpan informasi tanggal dan waktu pengiriman pesan

- **Nama Tabel:** laporan
- **Berkas Migrasi:** `database/migrations/ 2026_09_28_135735_create_laporan_table.php`
- **Status Commit:** [ ] Ter-commit

Nama Kolom	Tipe Data	Constraint / Relasi	Keterangan
Id_laporan	Primary key	tidak	Primary Key (ID unik laporan)
Periode	Varchar	tidak	Menyimpan informasi periode waktu laporan
Jumlah_pesanan	Integer	tidak	Menyimpan total angka jumlah pesanan dalam periode tersebut
Jumlah_pembayaran	Decimal	tidak	Menyimpan total nominal angka jumlah pembayaran (15 digit, 2 desimal)

4.1 Berkas Model di Repositori

TAHAP 4: IMPLEMENTASI MODEL ELOQUENT & RELASI

Jalankan `make:model` pada terminal proyek, lalu catat model yang dihasilkan beserta tabel yang diwakilinya:

No	Model (app/Models/)	Tabel	Relasi Utama	Commit
1	User.php	Users	hasMany(...)	[ ]
2	Pelanggan.php	Pelanggan	belongsTo / hasMany(...)	[ ]
3	Karyawan.php	Karyawan	belongsTo(...)	[ ]
4	Admin.php	Admin	belongsTo(...)	[ ]
5	Antrean.php	Antrean	belongsTo(...)	[ ]
6	Pesanan.php	Pesanan	belongsTo / hasMany(...)	[ ]

4.2 Model Induk (HasMany / HasOne)

- **Berkas:** app/Models/user.php
- **Status Commit:** [ ] Ter-commit

Implementasikan `$fillable`, `$casts`, dan method relasi pada model tersebut di repositori, lalu rangkum isinya di sini:

Properti dan relasi yang didefinisikan pada model ini:

Method / Properti	Jenis	Target / Isi	Keterangan
$fillable	Properti	nama, email, password, role	Menentukan kolom yang diizinkan untuk mass assignment
Pelanggan	Relasi	HasMany (Pelanggan::class)	Relasi one-to-many ke tabel pelanggan
Karyawan	Relasi	HasOne (Karyawan::class)	Relasi one-to-one ke tabel karyawan
Admin	Relasi	HasOne (Admin::class)	Relasi one-to-one ke tabel admin

4.3 Model Anak (BelongsTo)

- **Berkas:** app/Models/pelanggan.php
- **Status Commit:** [ ] Ter-commit

Implementasikan `$fillable` (termasuk foreign key) dan method `belongsTo()` pada model tersebut di repositori, lalu rangkum relasinya di sini:

Method Relasi	Target Model	Foreign Key	Keterangan
$fillable	-	`id_user`, `no_telepon`, `alamat`	Menentukan kolom yang diizinkan untuk mass assignment
User	User::class	Id_user	Relasi *belongsTo* yang menghubungkan data pelanggan ke pemilik akun user

4.4 Ringkasan Relasi Seluruh Model

Setelah semua model selesai, lengkapi tabel ringkasan berikut:

Model	Method Relasi	Target Model	Jenis Relasi Eloquent
User	Pelanggan	Pelanggan	hasMany
User	Karyawan	Karyawan	hasOne
User	Admin	Admin	hasOne
Pelanggan	User	User	belongsTo
Pelanggan	Pesanan	Pesanan	hasMany
Karyawan	User	User	belongsTo
Pesanan	Antrean	Antrean	belongsTo
Pesanan	Pembayaran	Pembayaran	hasMany
Pesanan	Pesan	Pesan	hasMany
Pembayaran	Pesanan	Pesanan	belongsTo
Pesan	Pesanan	Pesanan	belongsTo

Admin	User	User	belongsTo
Antrean	Pesanan	Pesanan	hasMany
Pesanan	Pelanggan	Pelanggan	belongsTo

TAHAP 5: FACTORY & DATABASE SEEDER

5.1 Berkas Factory & Seeder di Repositori

Jalankan `make:factory` dan `make:seeder` pada terminal proyek, lalu catat berkas yang dihasilkan:

No	Berkas	Jenis	Model / Tabel Sasaran	Commit
1.	database/factories/UserFactory.php	Factory	User / users	
2.	database/factories/PelangganFactory.php	Factory	Pelanggan/pelanggan	
3.	database/factories/KaryawanFactory.php	Factory	Karyawan/karyawan	
4.	database/factories/AdminFactory.php	Factory	Admin/admin	
5.	database/factories/AntreanFactory.php	Factory	Antrean/antrean	
6.	database/factories/PesananFactory.php	Factory	Pesanan/pesanan	
7.	database/factories/PembayaranFactory.php	Factory	Pembayaran/pembayaran	
8.	database/factories/PesanFactory.php	Factory	Pesan/pesan	
9.	database/factories/LaporanFactory.php	Factory	Laporan/laporan	
10.	database/seeders/UserSeeder.php	Seeder	User/users	
11.	database/seeders/PelangganSeeder.php	Seeder	Pelanggan/pelanggan	
12.	database/seeders/DatabaseSeeder.php	Seeder	Seluruh Tabel (Master Seeder)	

5.2 Rancangan Factory

- **Berkas:** `database/factories/UserFactory.php`
- **Status Commit:** [ ] Ter-commit

Implementasikan method `definition()` pada berkas factory di repositori (contoh helper: `fake()->name()`, `fake()->unique()->safeEmail()`). Rangkum pemetaan kolomnya di sini:

Alasan pemilihan helper `fake()` yang digunakan:

Kolom	Helper Faker	Alasan Pemilihan
Nama	fake()->name()	Menghasilkan nama lengkap acak yang realistis untuk pengguna
Email	fake()->unique()->safeEmail()	Menghasilkan alamat email acak yang aman, unik, dan tidak duplikat
Password	`bcrypt('password')` atau `Hash::make('password')`	Menghasilkan string password terenkripsi standar untuk pengujian
Role	fake()->randomElement(['admin', 'pelanggan', 'karyawan'])	Menentukan pilihan peran acak yang sesuai dengan sistem hak akses aplikasi

5.3 Rancangan Seeder

- **Berkas:** `database/seeders/DatabaseSeeder.php`
- **Status Commit:** [ ] Ter-commit

Seeder diisi pada repositori (data master statis lewat `Model::create([...])`, data acak lewat `Model::factory(n)->create()`). Catat sumber dan volume data dummy yang dihasilkan:

Model	Sumber Data	Jumlah Baris	Keterangan
`User`	`Model::create([...])`	3	Data statis akun utama awal
`Pelanggan`	Model::factory(n)->create()	10	Data acak dummy
`Karyawan`	Model::factory(n)->create()	5	Data acak dummy

5.4 Urutan Pemanggilan pada `DatabaseSeeder.php`

- **Berkas:** `database/seeders/DatabaseSeeder.php`
- **Status Commit:** [ ] Ter-commit

Catat urutan pemanggilan seeder pada berkas tersebut (urutan harus menghormati relasi induk → anak):

Urutan	Seeder Class	Tujuan Data
1.	UserSeeder::class	Mengisi data master induk akun pengguna
2.	PelangganSeeder::class	Mengisi data profil pelanggan yang berelasi ke tabel users
3.	KaryawanSeeder::class	Mengisi data profil karyawan yang berelasi ke tabel users
4.	AdminSeeder::class	Mengisi data profil admin yang berelasi ke tabel users
5.	AntreanSeeder::class	Mengisi data antrean layanan
6.	PesananSeeder::class	Mengisi data pesanan yang berelasi ke pelanggan dan antrean
7.	PembayaranSeeder::class	Mengisi data transaksi pembayaran yang berelasi ke pesanan
8.	PesanSeeder::class	Mengisi data pesan komunikasi yang berelasi ke pesanan
9.	LaporanSeeder::class	Mengisi rekapitulasi data laporan berkala

TAHAP 6: EKSEKUSI, VERIFIKASI & PENGUJIAN BASIS DATA

6.1 Jalankan Fresh Migration + Seeding

Jalankan perintah berikut pada terminal proyek (yang disalin ke lembar kerja hanya ringkasan log, bukan kode program):

PERINTAH TERMINAL (ARTISAN)

```
php artisan migrate:fresh --seed
```

Tempelkan ringkasan output log terminal sebagai bukti eksekusi:

AREA JAWABAN — SILAKAN TULIS DI SINI

```
2026_09_28_135154_create_users_table ..........................................89.98ms DONE
2026_09_28_135355_create_pelanggan_table ......................................207.77ms DONE
2026_09_28_135446_create_karyawan_table .......................................162.21ms DONE
2026_09_28_135512_create_admin_table ..........................................143.33ms DONE
2026_09_28_135538_create_antrian_table ........................................34.60ms DONE
2026_09_28_135600_create_pesanan_table ........................................263.06ms DONE
2026_09_28_135623_create_pembayaran_table .....................................138.58ms DONE
2026_09_28_135717_create_pesan_table ..........................................147.83ms DONE
2026_09_28_135735_create_laporan_table ........................................47.78ms
DONE.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
```

6.2 Tabel Checklist Verifikasi di DBMS

No	Nama Tabel	Status Migrasi	Jumlah Baris Data	Tipe Data Sesuai ERD
1	users	[ ] Berhasil	_____	[ ] Ya
2	Pelanggan	[ ] Berhasil	_____	[ ] Ya
3	Karyawan	[ ] Berhasil	_____	[ ] Ya
4	Admin	[ ] Berhasil	_____	[ ] Ya
5	Antrean	[ ] Berhasil	_____	[ ] Ya
6	Pesanan	[ ] Berhasil	_____	[ ] Ya
7	Pembayaran	[ ] Berhasil	_____	[ ] Ya
8	Pesan	[ ] Berhasil	_____	[ ] Ya
9	laporan	[ ] Berhasil	_____	[ ] Ya

6.3 Bukti Screenshot Verifikasi

No	Konten Screenshot	Nama File
1	Daftar tabel di DBMS	`screenshots/db_tables_list.png`
2	Contoh data dummy tabel master	`screenshots/db_data_master.png`
3	Contoh data dummy tabel transaksi	`screenshots/db_data_transaksi.png`

TAHAP 7: REFLEKSI DAN EVALUASI MANDIRI

1. Mengapa urutan timestamp file migrasi krusial saat mendefinisikan foreign key?

AREA JAWABAN — SILAKAN TULIS DI SINI

Laravel mengeksekusi migration berurutan **berdasarkan timestamp ASC** (dari yang paling tua / angka terkecil ke paling baru / angka terbesar). Anda bisa lihat daftar file di direktori migrations:...

Alasan 1: FOREIGN KEY hanya bisa mereferensikan tabel yang SUDAH ADA

Dalam SQL, Anda **tidak bisa** membuat CONSTRAINT FOREIGN KEY yang menunjuk ke tabel yang **belum dibuat**. Itu adalah aturan mesin database (MySQL/InnoDB, dsb).

Contoh nyata di proyek:

create_pelanggan_table.php baris 17:
PHP
```
$table->foreignId('id_user')
    ->constrained('users', 'id_user')
    ->onDelete('cascade');
```

Ini BERHASIL karena users (timestamp **135154**) dibuat **sebelum** pelanggan (timestamp **135355**).

Alasan 2: Rantai relasi multi-level harus terurut

Proyek Anda punya **rantai relasi bertingkat**:

users (induk tertinggi)
├──→ pelanggan
│ └──→ pesanan
│ ├──→ pembayaran
│ └──→ pesan
├──→ karyawan
└──→ admin

antrean ──────────────────────────→ (opsional ke) pesanan

laporan (tabel independen)
Lihat bagaimana create_pesanan_table.php membutuhkan **2 tabel induk** sekaligus:
PHP
```
$table->foreignId('id_pelanggan')
    ->constrained('pelanggan', 'id_pelanggan')   // ← butuh tabel pelanggan sudah ada
    ->onDelete('cascade');

$table->foreignId('id_antrean')->nullable()
    ->constrained('antrean', 'id_antrean')       // ← butuh tabel antrean sudah ada
    ->onDelete('set null');
```

Maka timestamp pesanan (**135600**) **HARUS** lebih besar dari:

- pelanggan (135355) ✅
- antrean (135538) ✅

Begitu juga pembayaran dan pesan harus berada **setelah** pesanan.

.....

.....

VI. RUBRIK PENILAIAN PRAKTIKUM MANDIRI

No	Komponen Penilaian	Bobot	Kriteria Capaian Maksimal
1	**Rancangan ERD & Integritas Relasi**	20%	ERD lengkap, kardinalitas jelas, sesuai skenario proyek kelompok
2	**Struktur File Migrasi & Constraint**	25%	Tipe data presisi, nullability tepat, foreign key cascade rapi
3	**Implementasi Model Eloquent**	25%	`$fillable` aman, method relasi dua arah terdefinisi benar
4	**Kualitas Factory & Seeder**	15%	Data dummy bervariasi, faker helper sesuai konteks realistis
5	**Verifikasi & Dokumentasi LKP**	15%	LKP terisi lengkap (tanpa salinan kode), log eksekusi bersih, screenshot valid
	TOTAL	**100%**	

Catatan penilaian: Komponen 2–4 (migrasi, model, factory & seeder) dinilai langsung dari source code pada repositori Git kelompok (lihat Bagian I), bukan dari lembar kerja ini.

VII. PEMETAAN KE SUB-CPMK

Tahap Kegiatan	Sub-CPMK yang Dicapai
Tahap 1 — Identifikasi entitas & ERD	S2
Tahap 2 — Konfigurasi database	S4
Tahap 3 — Migrasi & skema tabel	S4
Tahap 4 — Model Eloquent & relasi	S2, S4
Tahap 5 — Factory & Seeder	S4
Tahap 6 — Verifikasi basis data	S4
Tahap 7 — Refleksi mandiri	S2, S4

VIII. TATA CARA PENGUMPULAN

Deadline: Pertemuan 5 (awal perkuliahan)

Yang dikumpulkan:

1. **Repositori GitHub/GitLab** — push seluruh source code (migrasi, model, factory, seeder, `.env.example`) ke branch aktif proyek; pastikan tautan pada Bagian I dapat diakses dosen/asisten pengampu
2. **File lembar kerja ini** (format `.md` atau PDF) — terisi lengkap sesuai pengerjaan mandiri; **source code tidak dilampirkan pada lembar kerja**

3. **Folder `screenshots/`** di repositori — berisi ERD dan screenshot hasil verifikasi DBMS

Format nama file lembar kerja:

FORMAT / CONTOH PENAMAAN
`LKP04_[NIM]_[NamaLengkap].md`

Contoh: `LKP04_12345678_BudiSantoso.md`

Copyright © Edwin Hari Agus Prastyo, S.Kom., M.Kom. — Universitas Hasyim Asy'ari Tebuireng Jombang